

JAVA 17 OCP · YDS STUDY SERIES

Unit 06 · Technical Memory Notes · Class Design

Teknik İngilizceyi bağlam içinde öğren · kısa tekrarlarla kalıcılaştır

Bu dosya eksiksiz [çift dilli Chapter 6 notunu](#), inheritance ve object construction için tek bir zihinsel modelle tamamlar: compiler declaration'ı kontrol eder, constructor chain parent'a çıkar, execution parent'tan child'a iner, overridden instance method runtime object'e gider. Dil çalışması: [vocabulary](#).

Kapsam denetimi

Ana çift dilli kaynak inheritance, constructor output, overriding/hiding, abstract class ve immutability kurallarını tam kapsar. Bu hafıza notu şu teknik ayrımları birlikte tekrar ettirir:

- constructor declaration ile default constructor ilişkisinin netleştirilmesi,
- complete class/object initialization sırası,
- override/hide/redeclare/overload ayrım tablosu,
- checked exception ve access modifier override kuralları,
- immutable object için input/output defensive copy kontrolü.

1. Inheritance haritası

- Her class, **Object** hariç, tam bir direct superclass'a sahiptir.
- Java class'ları multiple class inheritance desteklemez.
- Bir class birden fazla interface implement edebilir.
- Subclass inherited accessible members'ı alır; constructors inherited değildir.
- **private** member kalıtımla alınmaz ve private method override edilmez. Nested sınıfların aynı top-level sınıf içindeki private erişim izni ayrı bir kuraldır; buna Unit 07'de dönülür.

```
class Animal {}
class Mammal extends Animal implements Comparable<Mammal> {
    public int compareTo(Mammal other) { return 0; }
}
```

Hafıza cümlesi: Bir class parent, çok interface contract.

2. **this** ve **super**

Form	Anlam
<code>this.field</code>	current object member
<code>super.field</code>	direct parent'taki visible member
<code>this(...)</code>	same class constructor delegation
<code>super(...)</code>	direct parent constructor call

`this(...)` ve `super(...)` yalnız constructor'ın ilk statement'ı olabilir; aynı constructor ikisini birden kullanamaz. Static context'te `this/super` object reference yoktur.

3. Constructor kuralları

Constructor:

- class ile aynı name'i taşır,
- return type taşımaz; `void` yazılırsa method olur,
- overload edilebilir,
- inherited veya overridden olmaz.

Compiler yalnız class hiçbir constructor bildirmediyse default no-argument constructor ekler. Her constructor'ın ilk statement'ında explicit `this(...)` veya `super(...)` yoksa compiler `super()` ekler.

```
class Parent {
    Parent(int value) {}
}

class Child extends Parent {
    Child() {
        super(1); // gerekli; Parent() yok
    }
}
```

Hafıza cümlesi: Constructor yazdıysan default gider; parent'ta boş kapı yoksa anahtarı sen verirsin.

4. Complete initialization order

Sınıfın initialization gerektiren ilk aktif kullanımında (örneğin `new Child()`):

- 1 parent static fields/blocks source order,
- 2 child static fields/blocks source order.

Her `new Child()` işleminde:

- 1 memory allocation ve fields için default values,
- 2 constructor chain'in parent'a kadar belirlenmesi,
- 3 parent instance fields/blocks source order,
- 4 parent constructor body,
- 5 child instance fields/blocks source order,
- 6 child constructor body.

```
static:    Parent → Child      (class başına bir kez)
instance: Parent → Child      (object başına bir kez)
class içi: fields/blocks source order → constructor body
```

`this(...)` zinciri aynı class initialization parçalarını tekrar çalıştırmaz; bir object için ilgili class'ın instance initializers'ı bir kez çalışır.

5. Method ilişkileri karar tablosu

İlişki	Ne değişir?	Seçim zamanı
Override	inherited instance method'a compatible implementation	runtime object type
Hide	inherited static method/field aynı adla saklanır	compile-time reference/context
Overload	same name, different parameters	compile time
Redeclare	private method gibi inherited olmayan member yeniden yazılır	ayrı declaration

Valid override checklist

Child method:

- 1 same signature taşınmalı,
- 2 same veya covariant return type kullanmalı,
- 3 access'i daraltmamalı,
- 4 broader/new checked exception bildirmemeli,
- 5 parent method `final`, `static` veya `private` olmamalı.

Unchecked exceptions bu checked-exception kısıtına tabi değildir.

```
class Parent {
    protected Number value() throws java.io.IOException { return 1; }
}

class Child extends Parent {
    @Override
    public Integer value() { return 2; }
}
```

Return `Integer` covariant, access daha geniş, checked exception daha dar olduğu için override geçerlidir.

6. Field ve static method hiding

```
class Parent {
    String name = "parent";
    static void call() { System.out.print("P"); }
    void run() { System.out.print("parent-run"); }
}

class Child extends Parent {
    String name = "child";
    static void call() { System.out.print("C"); }
    void run() { System.out.print("child-run"); }
}
```

`Parent ref = new Child();` için:

- `ref.name` → `Parent.name`,
- `ref.call()` → `Parent.call()`; instance üzerinden static çağrı tavsiye edilmez,
- `ref.run()` → `Child.run()`.

Hafıza cümlesi: Field ve static etikete, overridden method object'e bakar.

7. Abstract class ve concrete class

- Abstract class doğrudan instantiate edilemez.
- Abstract class zero veya more abstract method içerebilir.
- Abstract class constructor, fields ve concrete methods taşıyabilir.
- Abstract method body taşımaz.
- Abstract method `private`, `static` veya `final` olamaz; child implementation contract'ını imkânsızlaştırır.
- Concrete subclass inherited abstract methods'ın tümünü valid biçimde implement etmelidir.
- Class aynı anda `abstract final` olamaz.

```
abstract class Shape {  
    Shape() {}  
    abstract double area();  
    String label() { return "shape"; }  
}
```

8. Immutability checklist

Immutable object tasarlarken:

- 1 state fields `private` olmalı,
- 2 construction sonrası reassign edilmeyecek fields `final` olmalı,
- 3 setter veya state-changing method olmamalı,
- 4 subclass ile mutability eklenmesi engellenmeli (`final` class veya controlled hierarchy),
- 5 mutable input alınırken defensive copy yapılmalı,
- 6 mutable iç durum dışarı verilmeden korunmalı; koleksiyonun değiştirilemeyen görünümü mutable öğelerini kendiliğinden korumaz. Gerekirse öğelerin de koruyucu kopyası alınmalıdır.

```
final class Schedule {  
    private final java.util.List<String> days;  
  
    Schedule(java.util.List<String> days) {  
        this.days = java.util.List.copyOf(days);  
    }  
  
    java.util.List<String> days() {  
        return days; // List.copyOf sonucu unmodifiable  
    }  
}
```

Yalnız alanları `final` yapmak yeterli değildir; final referans mutable nesneyi gösterebilir. Örnekte `List.copyOf` liste değişikliğini engeller, `String` öğeleri zaten immutable'dır. Mutable öğe türünde yalnız liste kopyası almak derin değişmezlik sağlamaz.

Kapalı kitap karşılaştırması: Giriş listesini kopyalamazsan dışarıdaki değişiklik hangi alana ulaşır? Getter aynı mutable listeyi verirse constructor kopyası neden tek başına yeterli olmaz? [Soru 5](#) ve [Ünite 07 record karşılaştırması](#) ile kontrol et.

OCP için 20 saniyelik analiz sırası

- 1 Parent constructor gerçekten çağrılabilir mi?
- 2 `this()/super()` first statement mı ve cycle var mı?
- 3 Static sonra instance initialization sırasını parent→child yaz.
- 4 Same-name member instance method mı, static method mı, field mı?
- 5 Override checklist'in return/access/exception maddelerini uygula.
- 6 Abstract contract concrete class'ta tamamen karşılanmış mı?
- 7 Immutability'de mutable reference dışarı sızıyor mu?

Aktif hatırlama · Özgün çalışma soruları

- 1 Compiler ne zaman default constructor ekler?
- 2 `Parent p = new Child(); p.method()` hangi durumda Child method'una gider?
- 3 Override access modifier'ı daraltabilir mi?
- 4 `private final List` tek başına immutable state garanti eder mi?

Cevaplar

- 1 Class hiçbir constructor bildirmediğinde.
- 2 `method()` inherited instance method olarak valid override edildiyse.
- 3 Hayır; aynı veya daha geniş olmalıdır.
- 4 Hayır; list mutable olabilir ve reference sızdırılabilir.